digital

TECHNICAL MANUAL

PDP-11/05 PANEL REAWAKENING

Transformed into an Interactive Multi-Media Art Piece



This page intentionally left blank.

Table of Contents

Overview 5
Schematics 6
Panel Interface 6
Panel Control 7
Amplifier Mode Modification (Class AB) 8
Audio Signal Conditioning 9
Panel Switch Interface 10
Bill of Materials 11
Electronic Components 11
Structural and Mechanical Hardware 12
Prototyping 13
PCB Layouts 14
Panel Interface 14
Panel Control 15
Fabricated PCBs 16
Panel Interface 16
Control Board 17
Completed Circuit Boards 18
Panel Interface 18
Control Board 19
Board Assembly 20
Completed Assembly 23
Stand Logo 26
Firmware Overview 27
Cooperative Scheduler 27
Panel I/O 28
BT201 Driver 29
Audio Visualization (VU Meter) 29
Appendix I: Task Reference 30
Appendix II: BT201 Musings 33
Reading the Documentation 33
Command and Response Protocol 33
Task Architecture Notes 33
BLE/SPP Command Passthrough 34

Flash Provisioning 37
Flash Persistence Reference 37
Appendix III: The Saga of the Missing Key 39

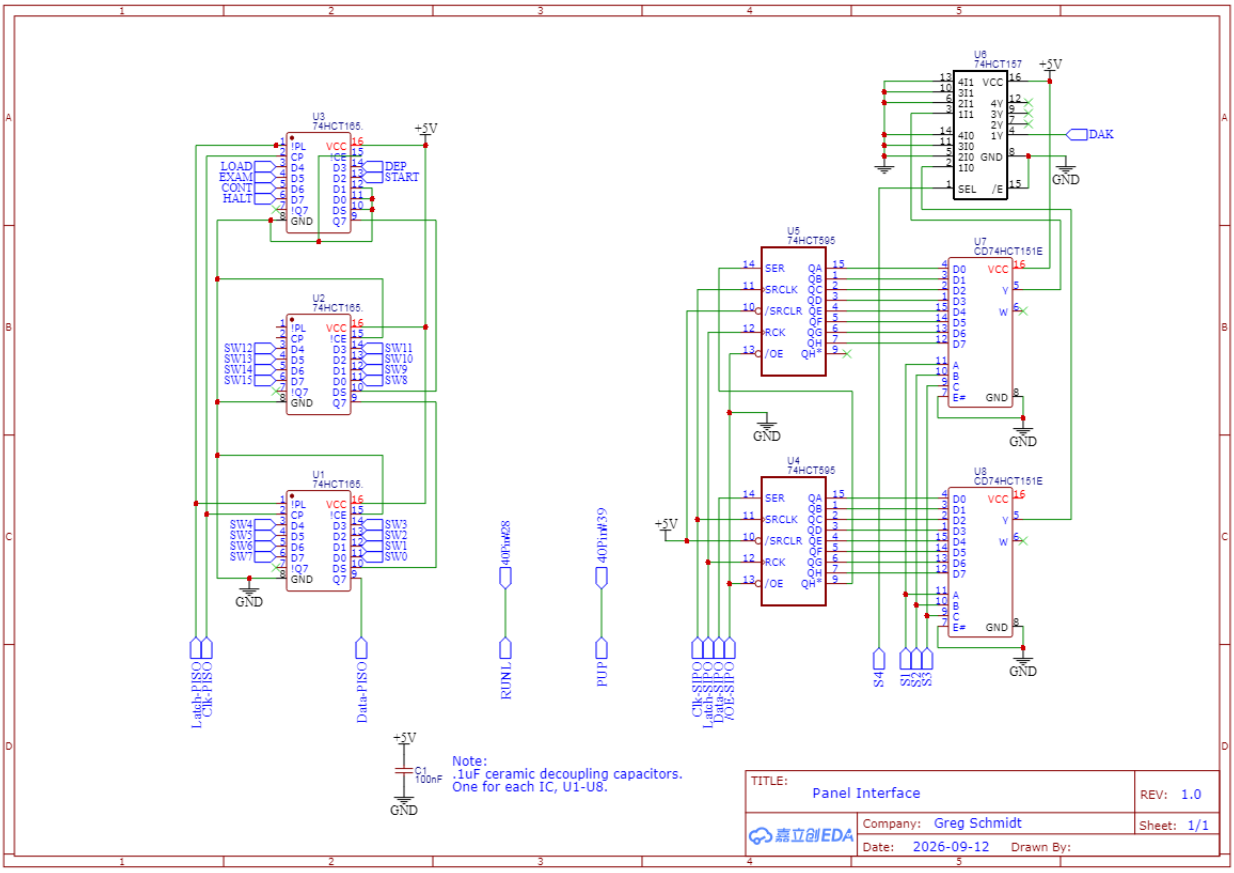
Overview

This document is the technical companion to the PDP-11/05 Panel Reawakening user manual. It covers the design and implementation details behind the piece: hardware architecture, firmware structure, and the reasoning behind key decisions. For operating instructions, please refer to the user manual.

The firmware source code, along with the schematics, documentation, and photos, is available in the project repository on GitHub: https://github.com/gschmidt958/pdp11-05-reawakening.

Schematics

Panel Interface



Description

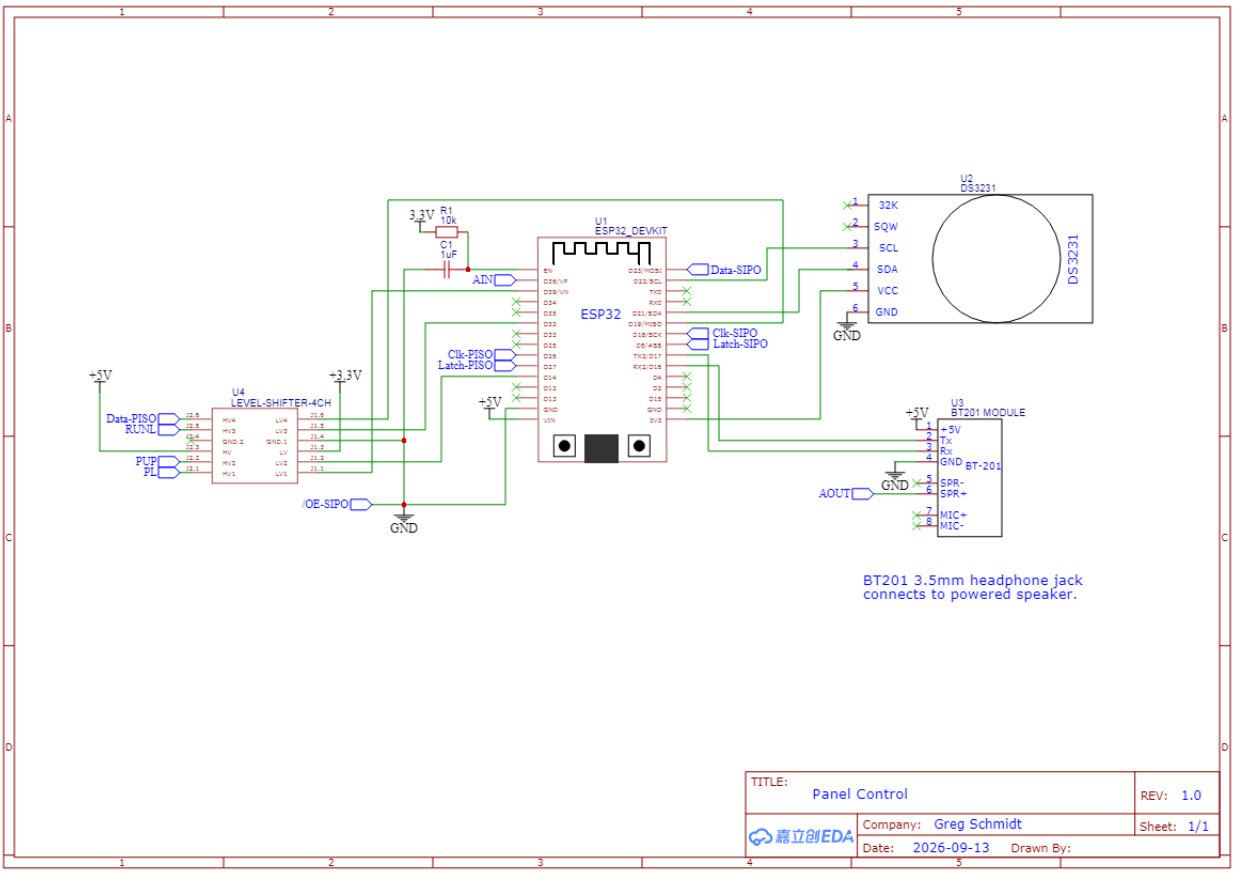
The interface board connects to the panel via a 40-pin ribbon cable. A standard 40-pin IDE ribbon cable can be modified for this purpose: remove (i.e. cut/file) the plastic protrusion on one end of the connector housing so it mates properly with the panel connector, which is unkeyed and expects a flat male connector. For noise immunity, the cable is wrapped in double-sided copper foil and insulated with wide electrical tape. A jumper wire is soldered to the foil and connected to ground. Photos of the finished connection are shown under Board Assembly, Panel Connector.

To read the 22 switches (16 address/data switches plus 6 sense switches), the interface uses three daisy-chained PISO shift registers.

For the LED display, the panel exposes 4 address lines, S1–S4. To determine the state of the 16 LEDs, the panel cycles through these address lines every 16 µs; the DAK signal returns the state of the addressed LED. The interface uses two daisy-chained 8-bit SIPO shift registers to form a 16-bit address/data register, along with two 8:1 multiplexers (74hct151) and a single 2:1 multiplexer (74hct157) combined to form a 16:1 multiplexer that routes the appropriate register bit (corresponding to the addressed LED by the 4 control lines) to DAK.

The interface uses CMOS HCT logic, which allows the modern 3.3V control signals to trigger the panel's legacy 5V TTL levels.

Panel Control



Description

The control board consists of an ESP32, a BT201 sound card, and a DS3231[1] real-time clock. It connects to the interface board, reading the switches from the PISO shift registers and writing LED state to the SIPO shift registers. BT201 audio is conditioned (see Audio Signal Conditioning) and then fed into the ESP32's ADC to support audio-reactive visualization on the LED display.

A 4-channel level shifter (U4) bridges the ESP32's 3.3V signals to the interface board's 5V logic for the Data-PISO, RUNL, PUP, and the chained /OE-SIPO lines.

RUNL is a direct digital output that drives the panel's RUN light. PUP (Power Up) is an output from the panel that indicates that the panel is ready.

A 10kΩ resistor from EN to 3.3V and a 1µF (105) capacitor from EN to ground are added. This is from the ESP32 datasheet. It is required to attain a reliable boot sequence.

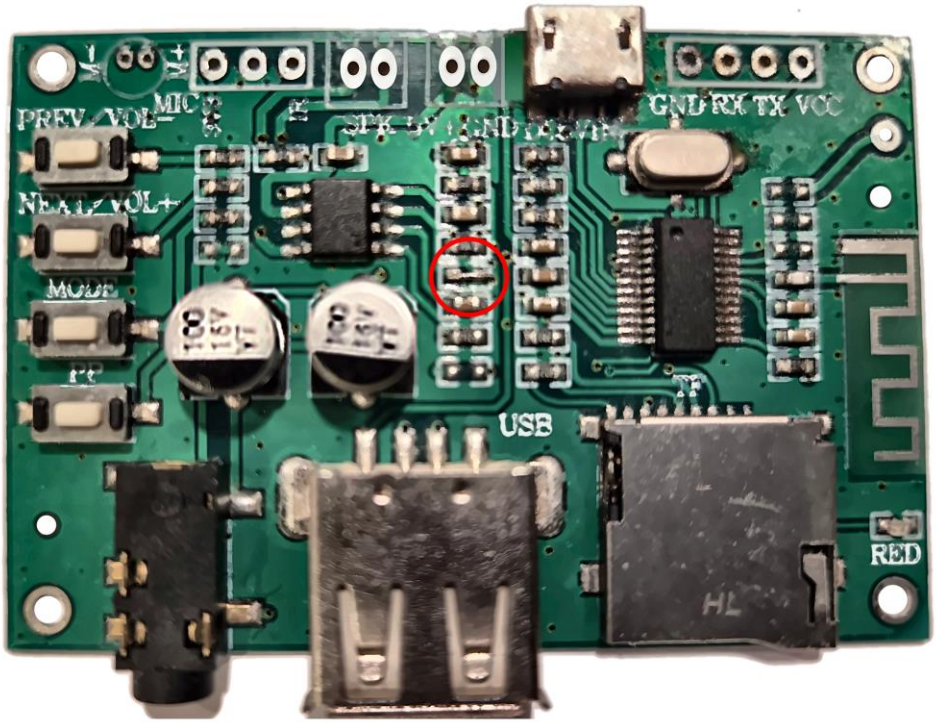
[1] Many common DS3231 breakout boards include an integrated trickle-charging circuit designed for rechargeable LIR2032 cells. If using a standard CR2032 coin cell, this onboard charging path must be disabled—typically by removing the series charging resistor or diode—to prevent hazardous reverse-charging and battery damage (see https://electronics.stackexchange.com/questions/134725/ds3231-module-circuit-teardown).

Amplifier Mode Modification (Class AB)

The BT201's audio output is driven by an HAA2018 power amplifier (the 8-pin chip on the board). Out of the box, the HAA2018 operates as a class D amplifier. Grounding its mode pin (pin 3) switches it to class AB operation, which is the preferred mode for sampling via the ESP32's ADC.

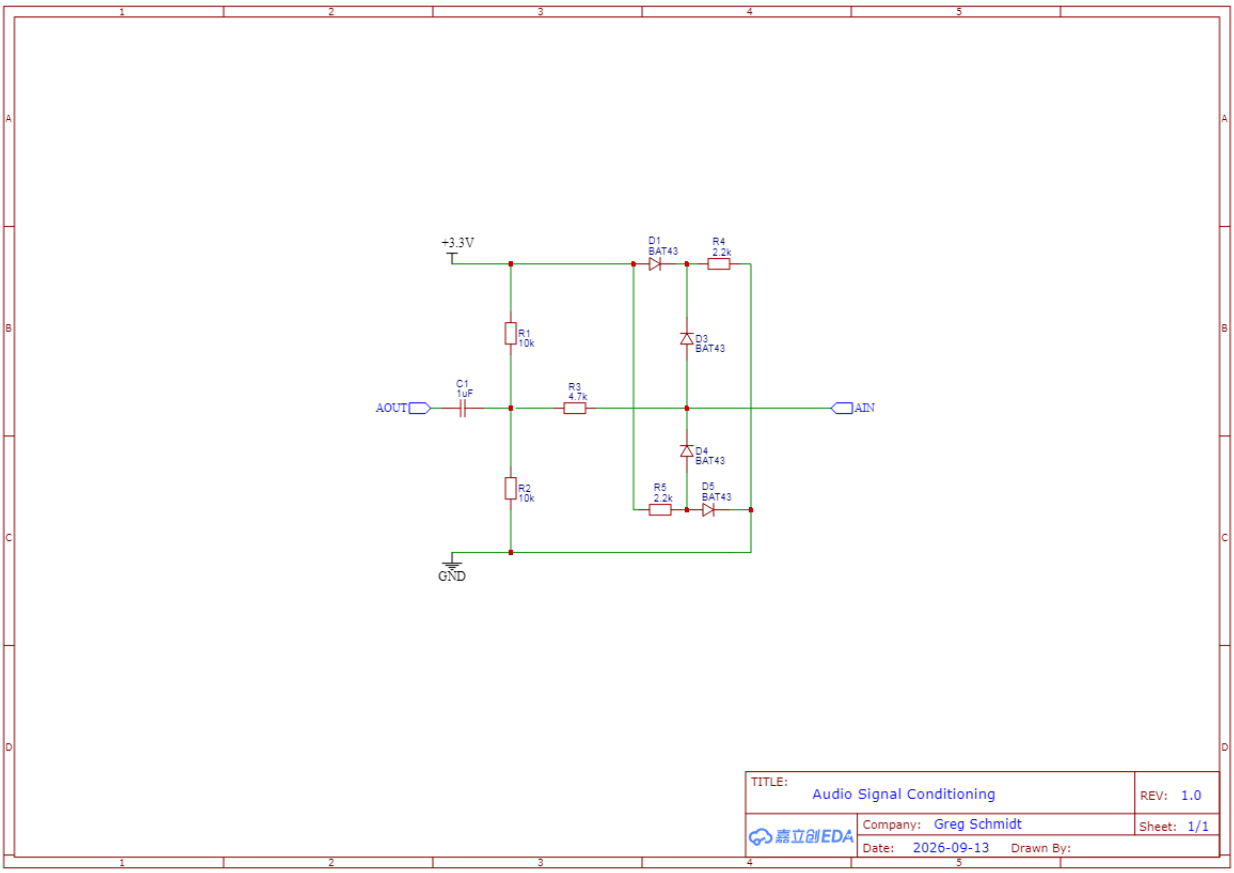
This modification is required: add a single jumper from pin 3 to ground, circled in red below.

Note: this jumper is an undocumented feature of the BT201 that was discovered by reading the HAA2018 datasheet.



BT201 board with the class AB jumper (HAA2018 pin 3 to ground) circled in red.

Audio Signal Conditioning

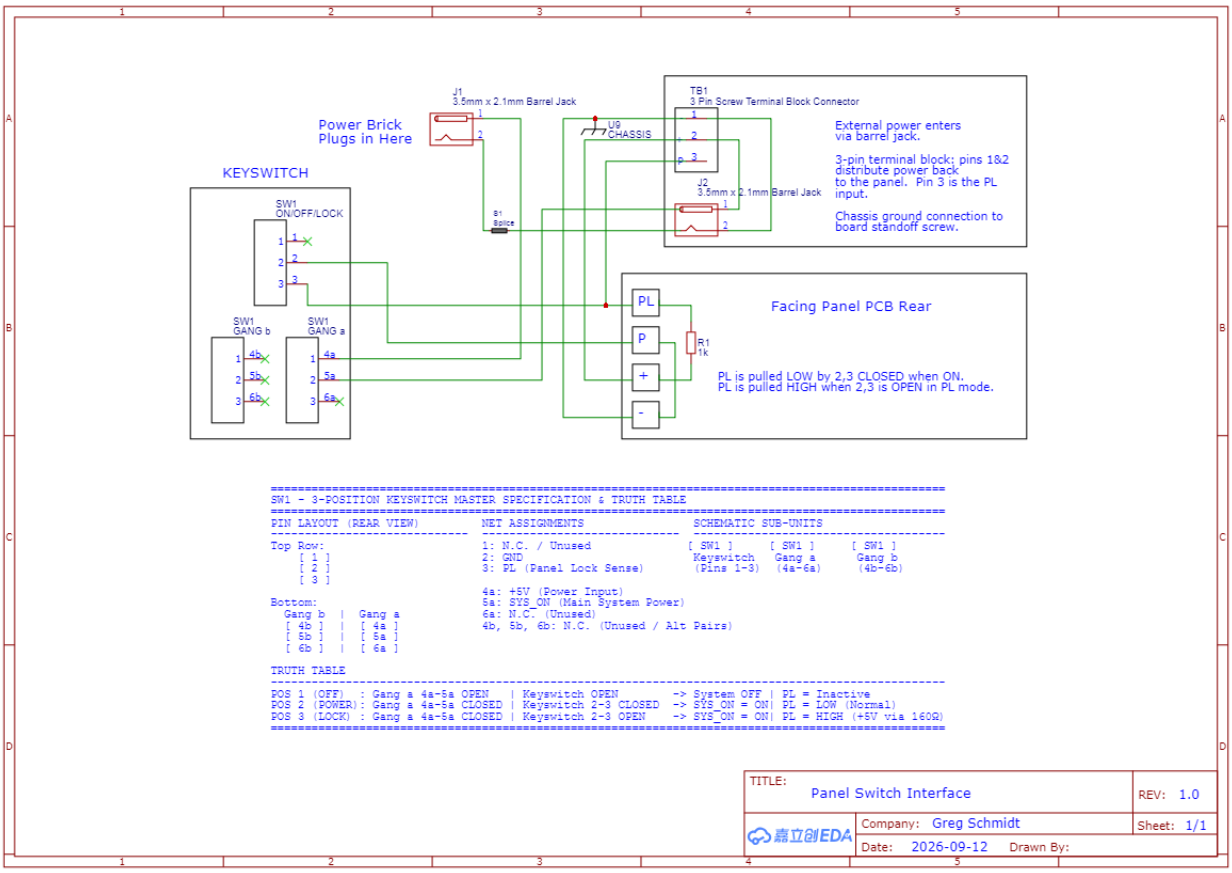


Description

The BT201 audio signal is taken from SPK+, one side of the BTL output of the HAA2018 power amplifier on the BT201 board. Using one side of the bridge intentionally halves the signal voltage, a 6 dB voltage reduction. The BT201 also provides stereo output on its 3.5 mm jack, however that is left unused because the mono signal from SPK+ suits the panel's single VU style bar display and avoids having to mix the stereo channels. The signal is capacitively coupled into a mid-rail bias network, and a series resistor reduces its strength further. Two Schottky clamp diodes protect the ADC input. A diode conducts only once the signal exceeds its forward drop, so an overvoltage or undervoltage from the AC waveform could still potentially reach the ADC before the clamp acts. To limit this, the clamp diodes return to two stiff "margin rails" instead of to Vcc and ground directly. The upper rail sits one diode drop below Vcc and the lower rail sits one diode drop above ground. The rail diodes and the clamp diodes are identical, so their forward drops cancel. The input is therefore held close to the supply rails, regardless of the exact forward voltage, which varies with temperature and current.

For the ADC to see a suitable signal, the BT201's amplifier should first be switched from its default class D mode to class AB; see Amplifier Mode Modification (Class AB), above.

Panel Switch Interface



Description

External power is plugged into the barrel jack and routed to the panel power switch, which in turn is routed to the barrel jack on the interface board, allowing the interface board to be switched on via the keyswitch. Via the 3-pin connector, the interface board routes switched power back to the panel. The panel lock position contact is also routed back to the control board so it can detect panel lock mode.

Bill of Materials

Electronic Components

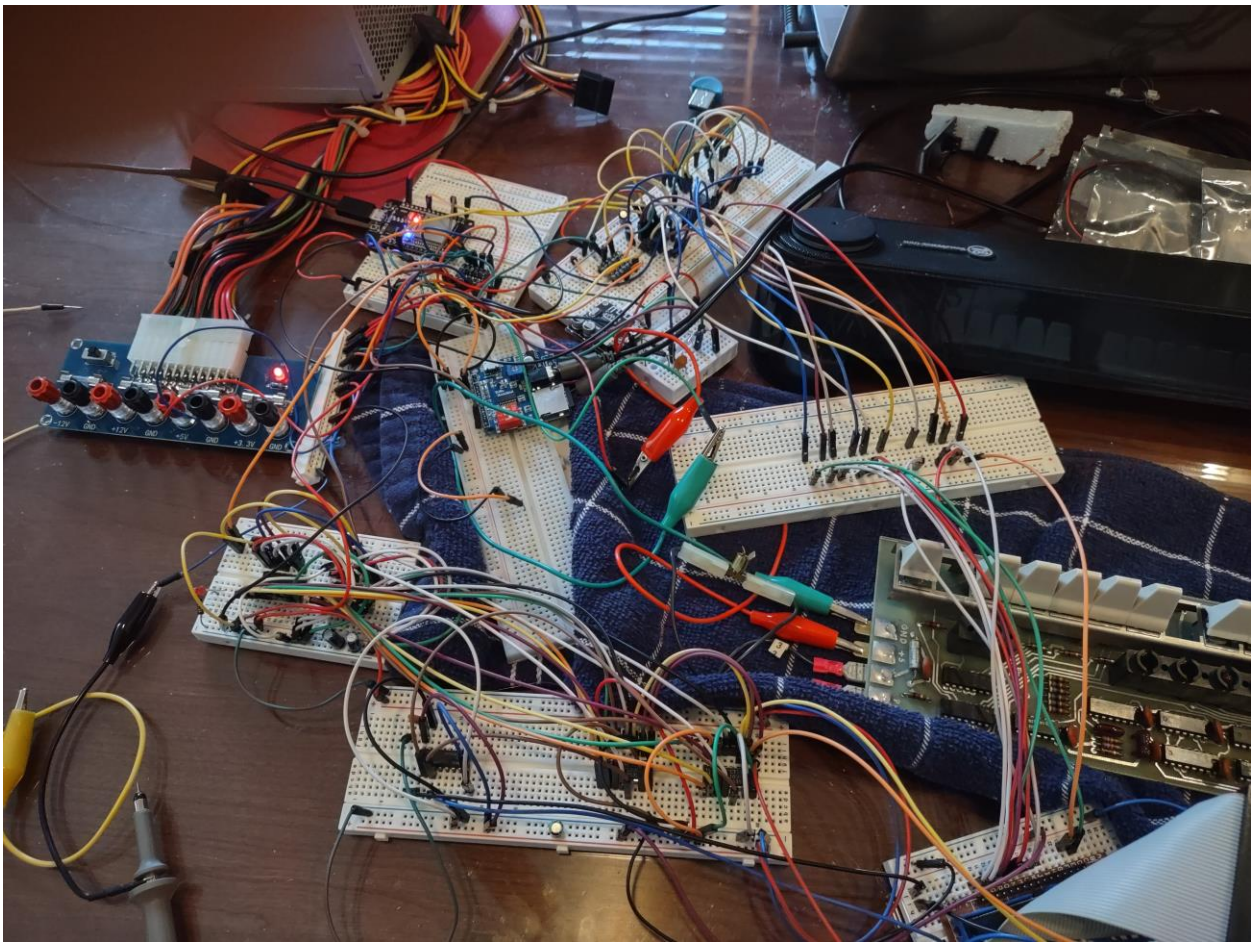
Qty	Item	Notes
1	1µF 63V EPCOS box film capacitor, ±10% (105)	Analog coupling cap
3	10kΩ 1/4W resistor	Analog bias / ESP32 EN pin
1	4.7kΩ resistor	Analog section series resistor
2	2.2kΩ resistor	Analog section margin rails
1	1µF ceramic capacitor (105)	ESP32 EN pin
4	BAT43 Schottky diode	Analog section clamping
1	40-pin female-to-female ribbon cable	A modified IDE cable can be used (see text)
1	40-pin mail connector housing	These can be purchased or salvaged from vintage PC motherboards.
1	2.54mm x 13mm header pins	Sufficient quantity for two rows of 40 pins.
1	2.1mm DC barrel jack (breadboard-friendly, DC005)	
1	3-pin 5.08mm pitch right-angle PCB screw terminal block.	Green, Phoenix-style.
1	100µF 16V electrolytic capacitor	Power decoupling cap (near barrel jack), low freq.
1	74HCT157	2:1 MUX
2	74HCT151	8:1 MUX
3	74HCT165	PISO shift register
2	74HCT595	SIPO shift register
9	0.1µF ceramic capacitor	Power decoupling caps, high freq. (one for each IC, one near barrel jack)
1	DS3231 RTC module	
1	BT201 Bluetooth audio module	Requires class AB jumper (see Amplifier Mode Modification)
1	ESP32-WROOM-32 development board	
1	4-channel logic level converter	I2C style
1	Micro SD card	40GB
1	3.5mm audio cable	Connects BT201 headphone output jack to powered speaker input.

Structural and Mechanical Hardware

Qty	Item	Notes
1	Apple Cinema Display stand	Salvaged component
4	10-32 x 2.5" stainless steel Phillips pan head machine screw	
4	10-32 stainless steel Kep nut	
4	#10 stainless steel flat washer	
16	6-32 x 3/8" machine screw	PCB mounting to standoffs
8	Standoffs (accept 6-32 x 3/8" machine screws on both ends)	My standoffs were custom made on a small lathe from aluminum stock, .28" dia. Central standoffs are .54" in length, outer standoffs are .61" in length. Dimensions are not critical as long as the boards remain a constant height from the backplate.
8	#6 stainless steel multi-tooth star lock washer	Back panel to PCBs via standoffs

Prototyping

Before committing to a PCB layout, the interface board's shift-register and multiplexer logic, along with the control board's audio conditioning and level shifting, were validated on breadboards. The control board originally paired a DY-SV5W SD card audio module with a separate BT 5.0 stereo receiver module. Since only one audio source could be routed to the ADC for visualization, the Bluetooth source was chosen. However, after discovering and replacing the pair with the BT201, which combines both functions into a single audio output, both SD card and Bluetooth audio could feed the ADC directly without the complexity of adding an analog switch.

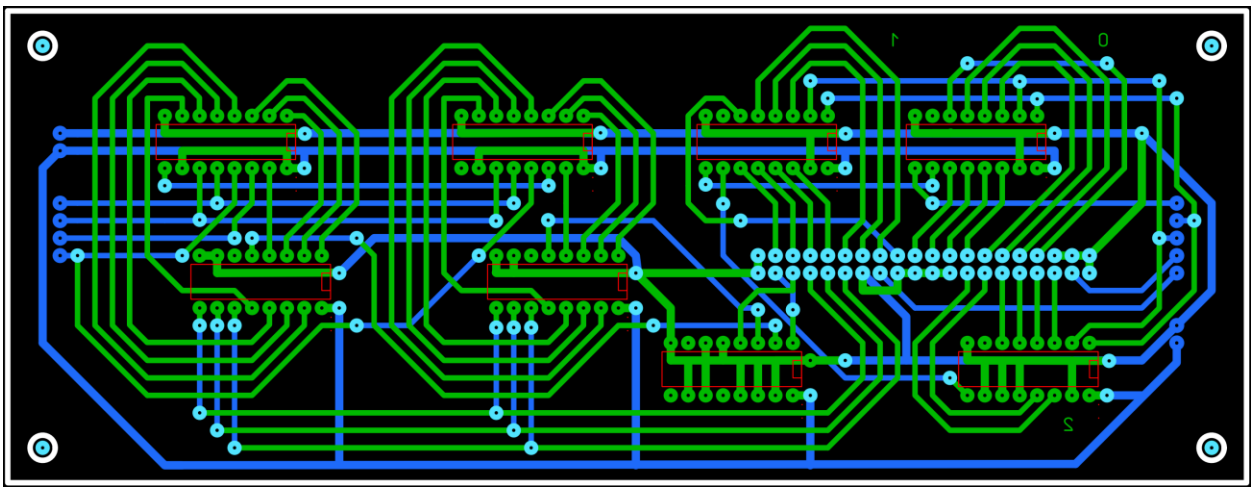


The early prototype was a mess of breadboards!

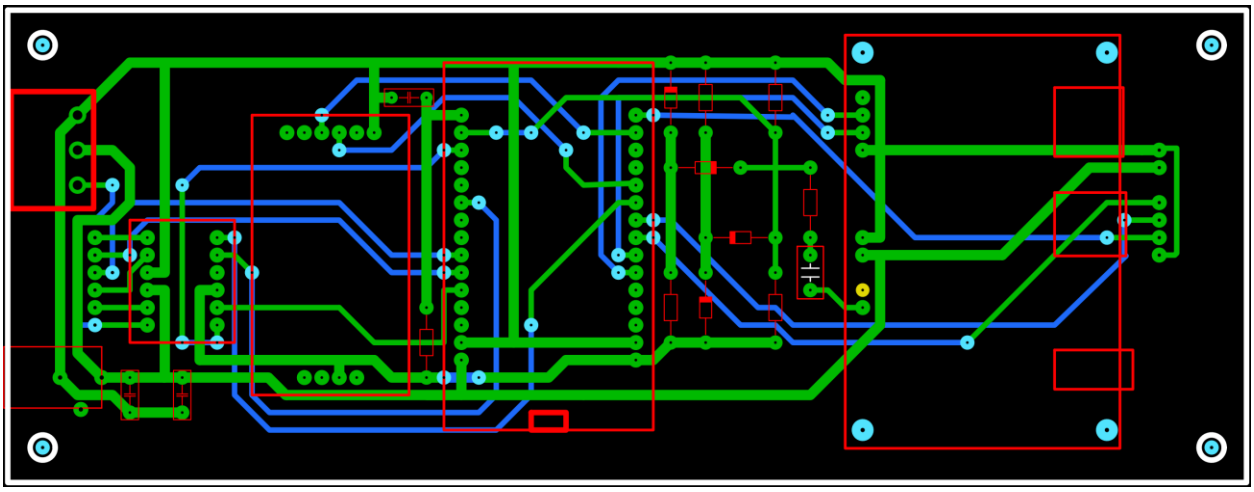
PCB Layouts

Both boards are double-sided.

Panel Interface



Panel Control



Description

The control board's left and right edge connectors carry the following signals, top to bottom:

Left Connector

Signal
Data 165
Latch 165
Clock 165
RunL
Pup
Minus
Plus

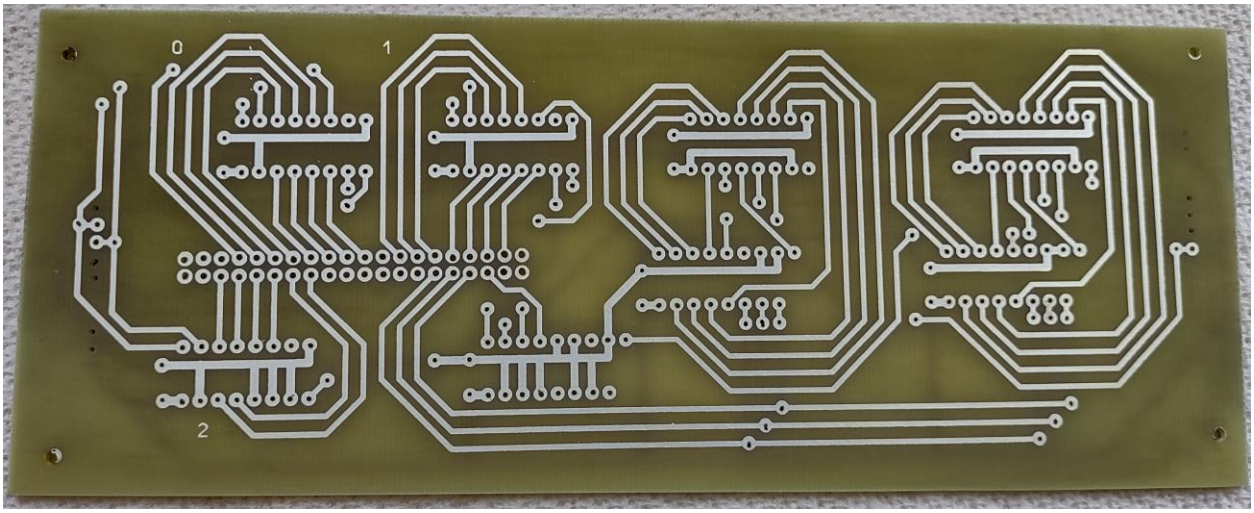
Right Connector

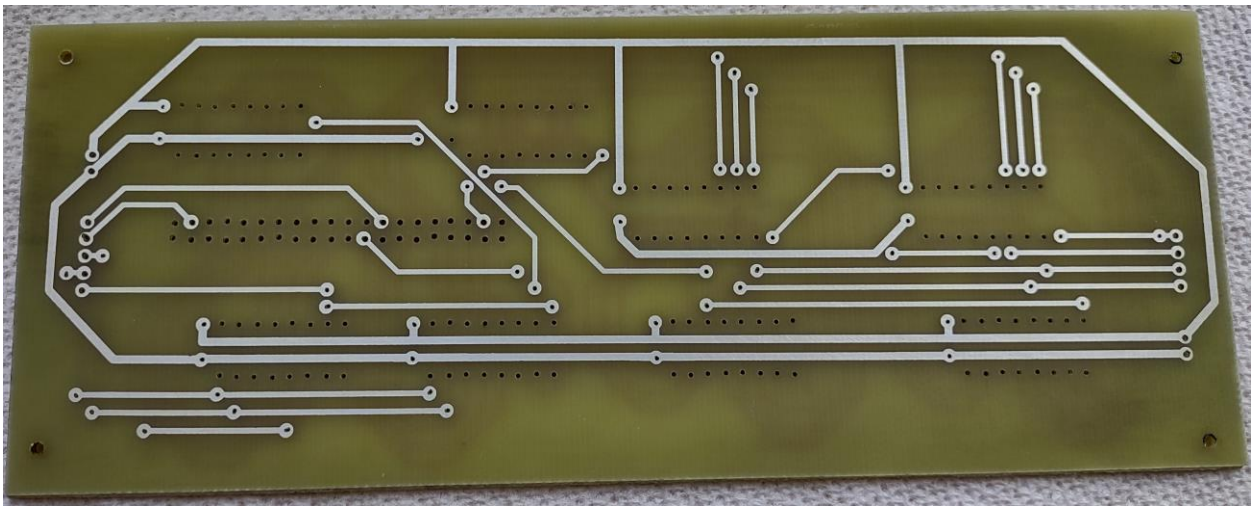
Signal
Minus
Plus
Latch 595
Clock 595
Data 595
/OE (output enable)

Fabricated PCBs

These boards were etched, drilled, and tinned at home.

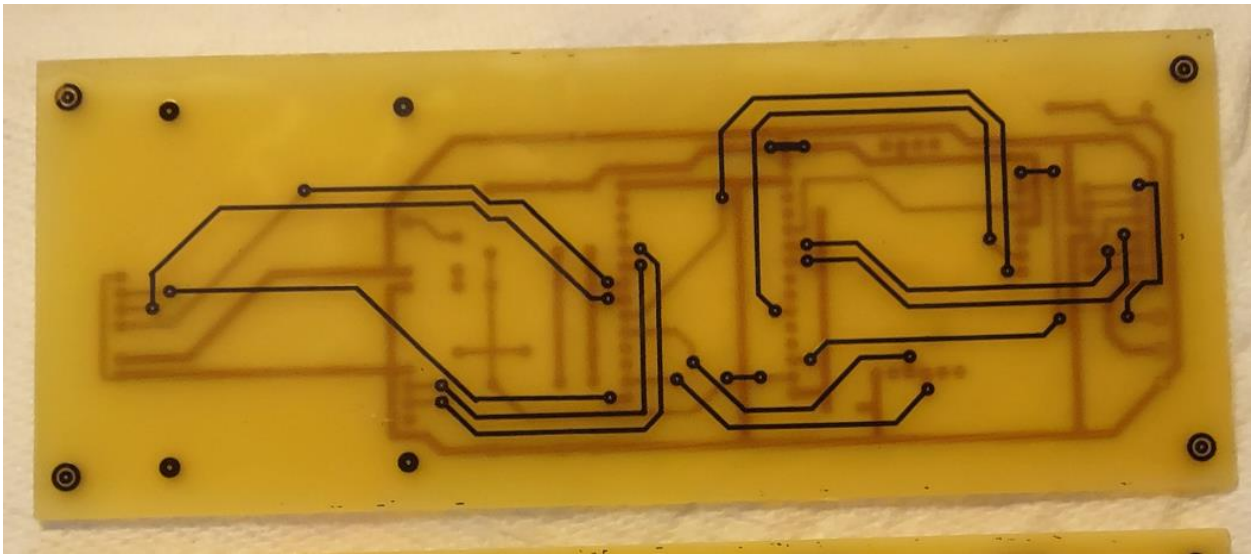
Panel Interface

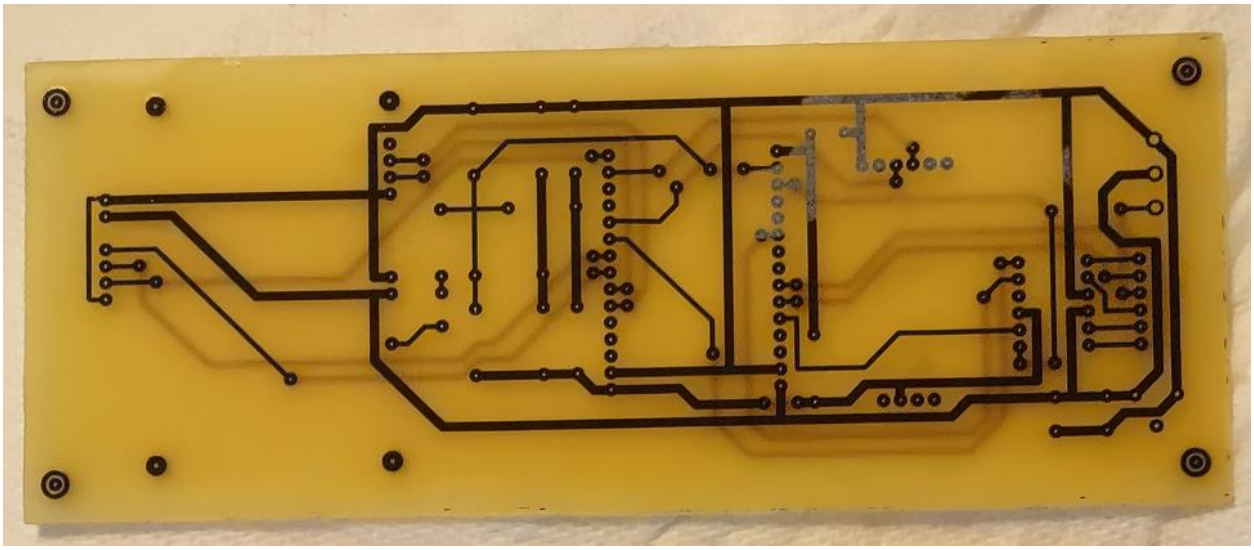




Control Board

These boards are shown prior to tinning.

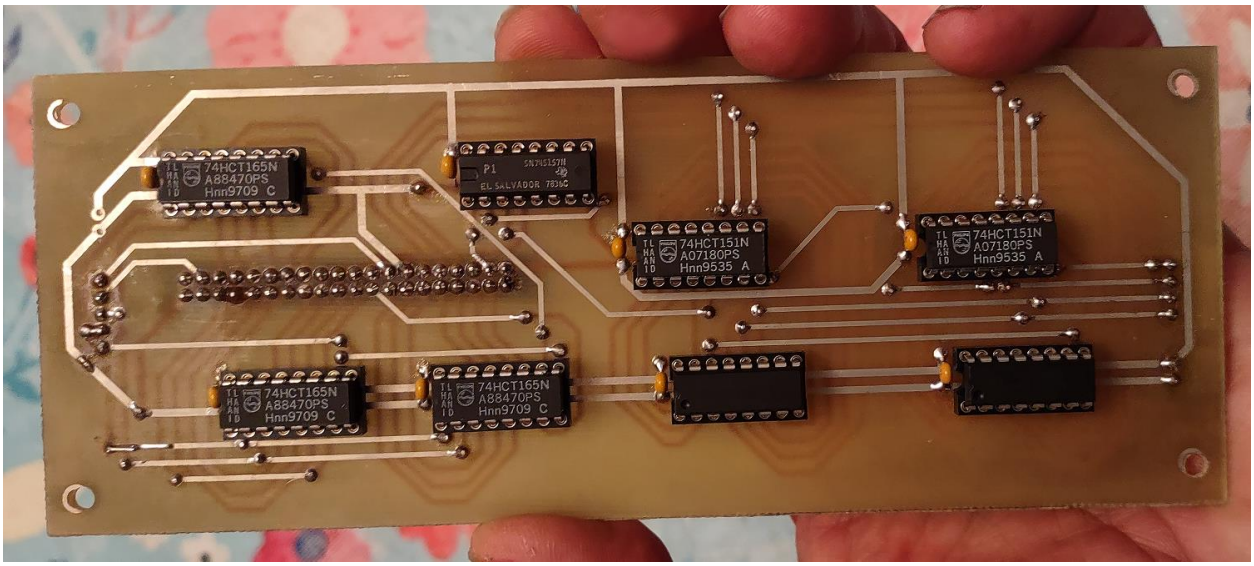




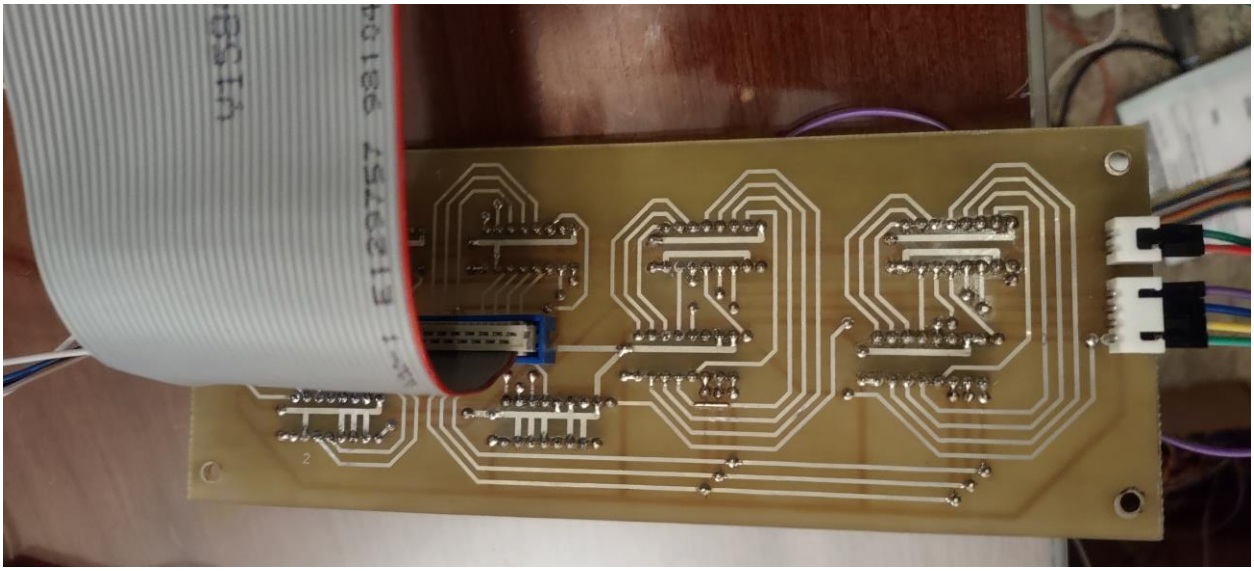
Completed Circuit Boards

Panel Interface

This board has been populated and is ready for installation.

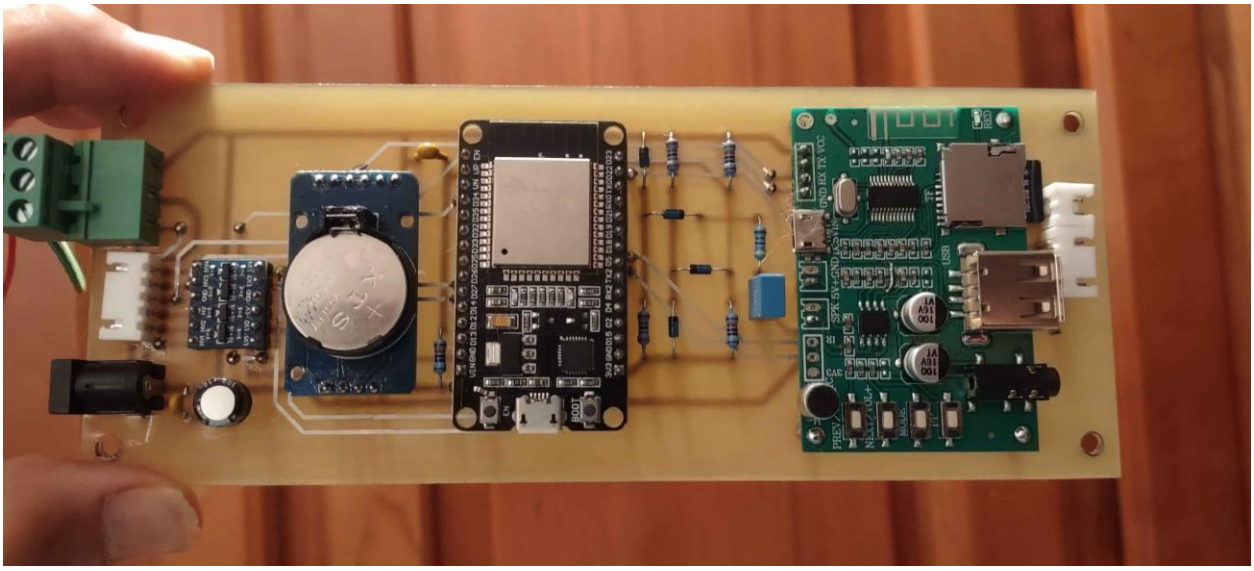


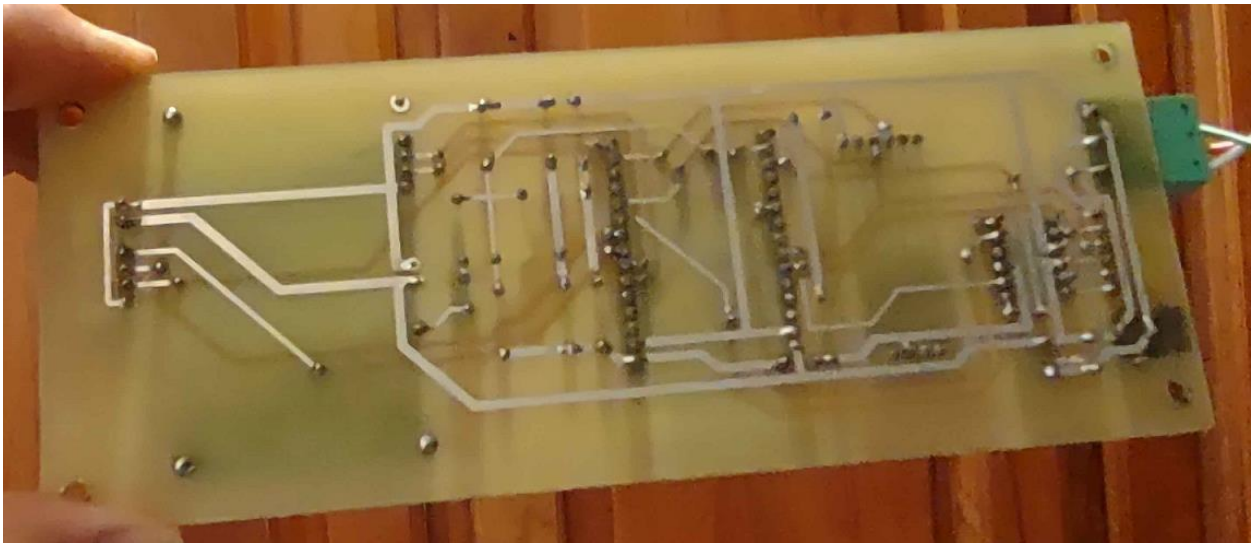
Bottom side of board showing the 40 pin ribbon cable and connector. Both the cable and connector housing were salvaged from an old PC motherboard with IDE peripherals. To connect the panel interface board to the control board, JST edge connectors were used for reliability. Until permanent JST style ribbon cable connectors were fashioned, Dupont style connectors were temporarily used to connect the two boards.



Control Board

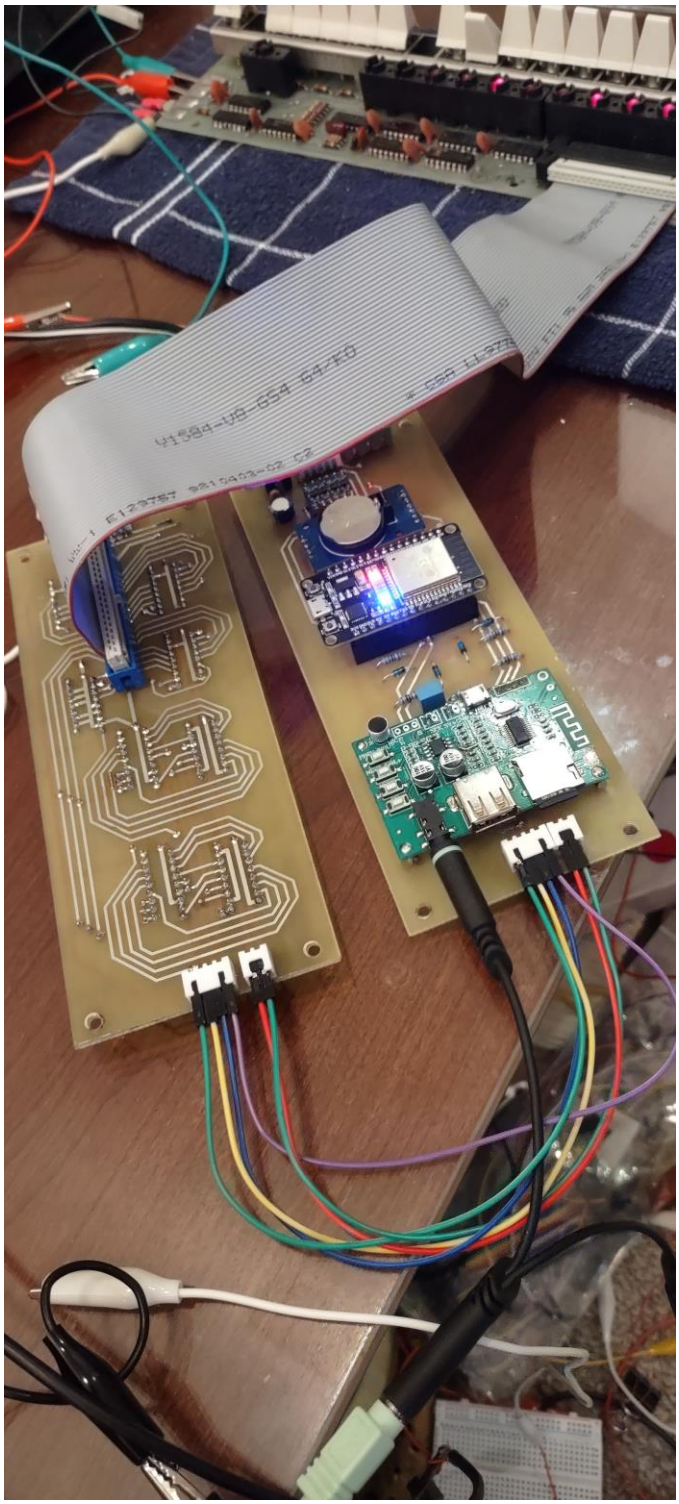
The control board has been populated and is ready for installation. From left to right: power and signal connections, level shifter, DS3231 clock module, ESP32, analog section, BT201 sound card, additional signal connections. JST, barrel jack, and Phoenix style connectors were used.





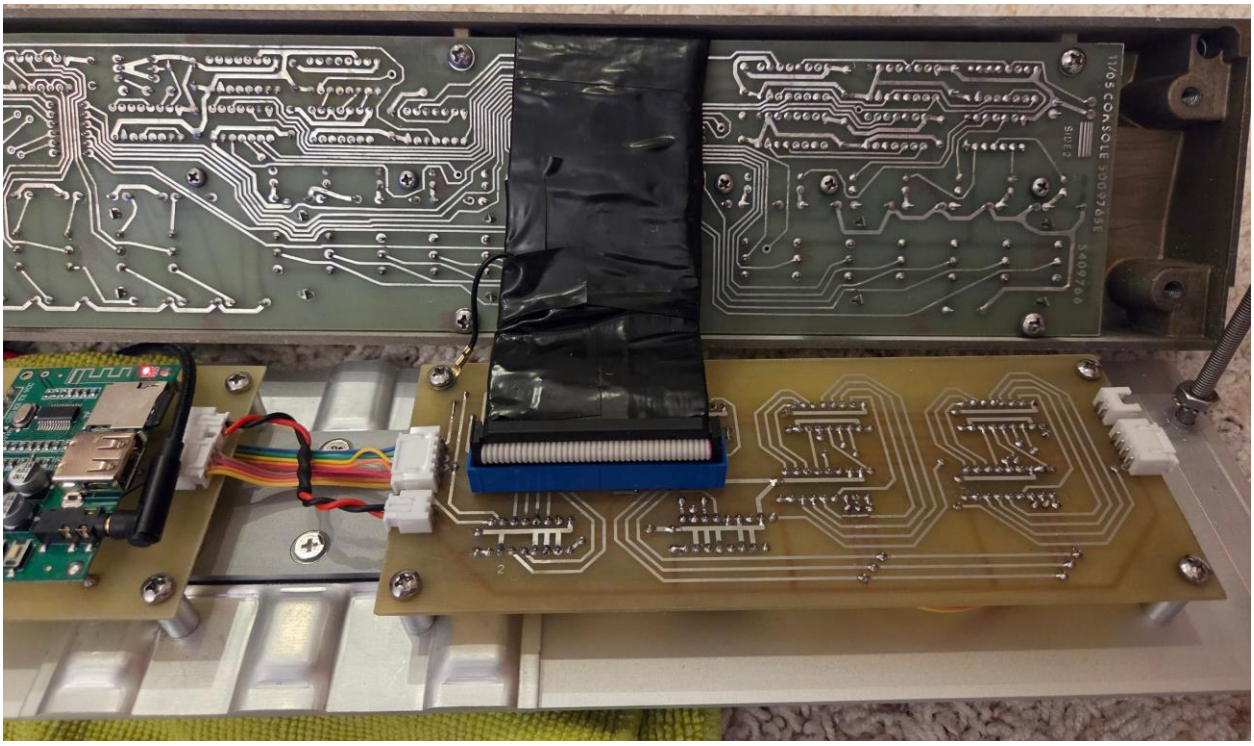
Board Assembly

The interface and control boards are shown temporarily wired together on the bench.

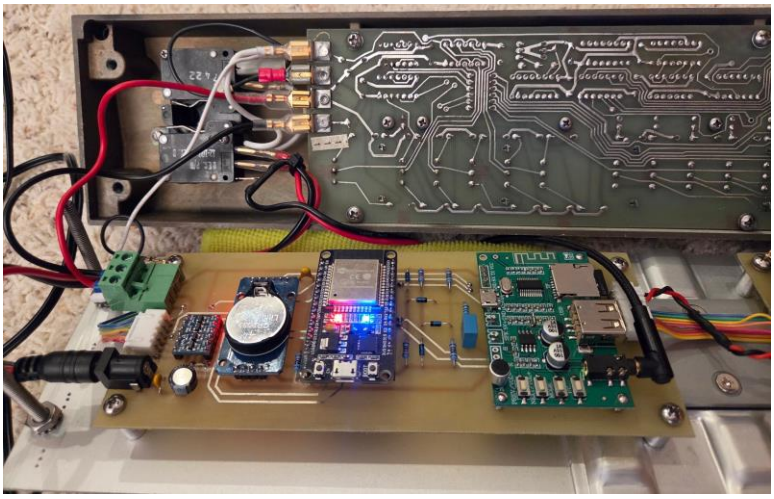


Both boards now attached to the back plate (cut from the back of the aluminum monitor stand) via standoffs and 6-32 machine screws. A sabre saw with a metal cutting blade was used to cut the back plate, then smoothed with sandpaper. Sharp edges were gently rounded with a file. The back plate dimensions are 18.5"L x 3.75"W. The width is not critical, and may be reduced further to make the back plate even less noticeable. The standoffs were machined on a mini lathe.

The two boards are now connected via ribbon cable and JST connectors. The 40 pin data cable, power and switch wiring, is also in place.



Service view with the panel opened. The 40-pin data cable runs from the interface board to the panel, shielded with copper foil and black electrical tape. The ground jumper for the foil is visible at the left edge of the tape.



Rear of the panel with the control board below, showing the panel switch and power wiring along with the control board's Phoenix connector and barrel jack.

Panel Connector

The interface board's 40-pin ribbon cable plugs into the Berg connector on the back of the panel. The connector is unkeyed, so the plastic protrusion on the cable's connector housing was removed to let it seat flat (see Panel Interface under Schematics).



Modified cable end seated flat in the panel's Berg connector.

Completed Assembly

The finished piece, powered on.



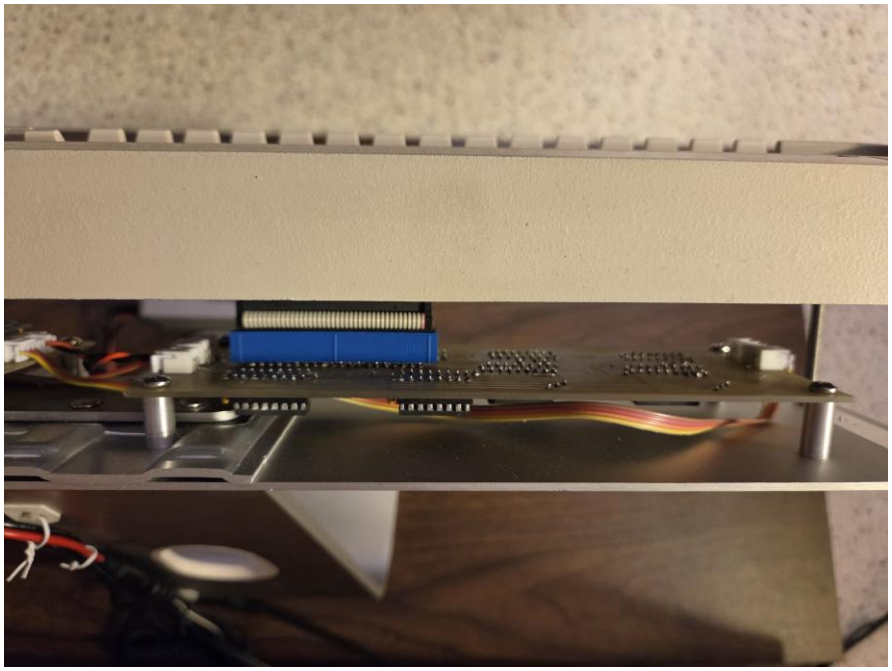
The completed PDP-11/05 panel, powered on and now reawakened, sitting on its Apple Cinema Display stand with the accompanying Bose SoundLink Mini.



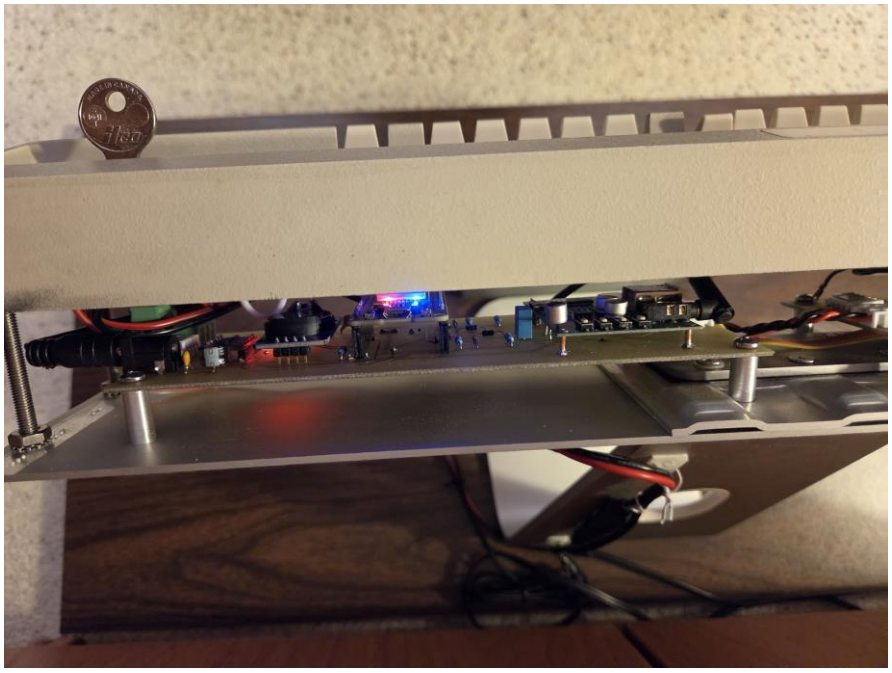
Top view showing the interface board (left) and control board (right) mounted on the back plate behind the panel.



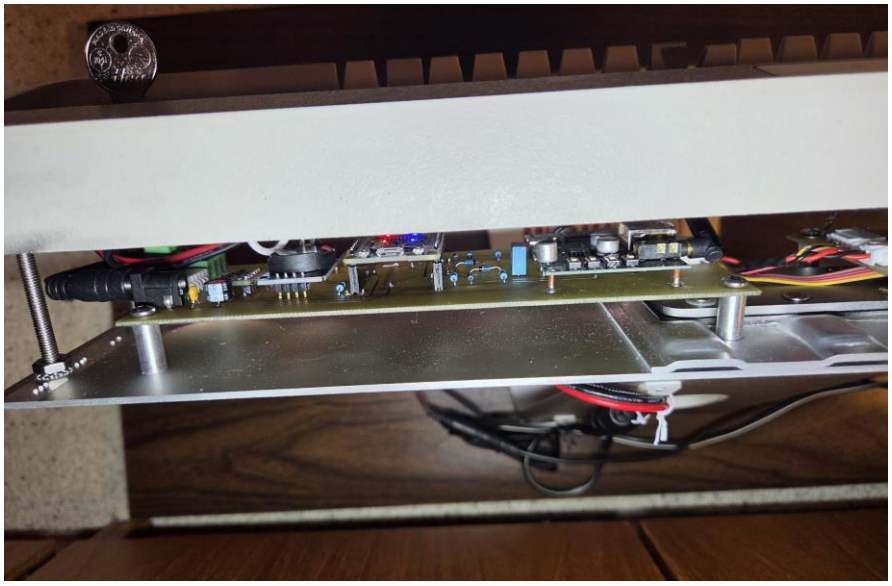
Rear view showing the back plate mounted to the stand, with the power cable routed vertically down the stand. Adhesive pads which accept wire ties were used to secure the cable.



Interface board on its standoffs, showing the 40-pin data cable connector and ribbon cable connections.



Control board with the DS3231 real-time clock, ESP32, and BT201 sound card.



Side view of the mounted control board.

Stand Logo

A reproduction "digital" logo is applied to the front of the stand base. Scale the logo to roughly 4-3/8" wide. For best results, print it on a laser printer, either on clear sticker stock or on transparency film adhered with double-sided tape. If using a transparency, the toner should be on the bottom side (flip the image horizontally before printing). Position the lower edge of the sticker approximately 1-3/16" in from the front edge of the stand base and centered.



Firmware Overview

The firmware runs as a set of cooperative tasks scheduled by a lightweight cooperative round robin scheduler. Tasks are roughly categorized as system and application tasks. System tasks are those considered foundational to the operation of the panel. The primary system tasks responsibilities are:

- Panel I/O - responsible for scanning the switches, and the display task that drives the LED display.
- UI - stops the current task, and starts a new one at the user's request based on the input selection from the panel switches. Also, monitors the panel lock keyswitch input.
- BT201 - A pair of tasks, consisting of a transmit task, and a receive task, along with a custom driver, which services the BT201 Bluetooth/MP3 module.
- "blink" - blinks the ESP32's onboard LED at 2 Hz serving as a "heartbeat".
- VU meter - drives audio-reactive LED displays.

Additional, application oriented tasks rely on many of these foundational tasks. See Appendix I for the complete task list. The complete source code is available in the project repository at https://github.com/gschmidt958/pdp11-05-reawakening.

Cooperative Scheduler

Each unit of work is a Task with a name and a period. The scheduler advances one active task per call to TaskScheduler::Run(), which is invoked continuously from the main loop. When a task comes due, the elapsed time since the task previously ran meets or exceeds the task's period (with 1ms resolution), and the task's Execute() method is called. The task runs to completion, and control is then returned back to the scheduler. Therefore tasks execute cooperatively without preemption. If a task must wait for an external event (e.g. audio track completion, switch closure, timer, etc.) it typically implements a state machine so that it can return control and poll for the event completion during subsequent executions of the task. Tasks move between active and inactive lists at runtime via the scheduler's StartTask(task) and EndTask(task) methods, which is how the panel switches from one mode to another. Each task also has lifecycle methods:

- Setup() - runs once, when the task is first added to the scheduler.
- Init() - runs each time the task is started.
- Cleanup() - runs when EndTask() is called on the scheduler. Performs any necessary cleanup required by the task prior to exiting.
- (Shutdown()/ShutdownComplete()) - Allows a parent task to ask a child task (i.e., a task the parent started) to wind down first. This is used, for example, to let audio finish before a mode actually stops running, so playback never bleeds into the next mode. If the parent task is requested to stop, it calls Shutdown() on the child task. Once the child task has completed, it calls ShutdownComplete() on the parent task. The parent task now exits. It's effectively a handshake which supports performing an orderly shutdown during a mode change.

Panel I/O

SwitchScanTask() reads the interface board's PISO shift registers each cycle, one group for the sixteen toggle-style address/data and mode switches (SwitchRegister) and one for the six sense switches (SenseSwitches). Debounce filters guard against the mechanical bounce of the panel's non-spring-loaded toggle switches, the address/data/mode switches and the ENABLE/HALT toggle, requiring a stable reading across a configurable number of scans before a change is accepted. Once accepted, the change is fired to any subscribed task through a small callback list managed by Subscribe()/Unsubscribe(), so downstream logic reacts to switch and sense-switch changes without further polling. BLE command support builds on this same event path: unsolicited $ commands arriving over the BT201's Bluetooth passthrough can set or clear individual SwitchRegister bits, pulse one of the momentary sense switches, toggle ENABLE/HALT, or set the panel-lock keyswitch state, each one producing the same subscriber callback a physical switch press would, so the panel can be operated remotely. On BLE command disconnect, switch input reverts back to the debounced physical state so the panel reflects the real switches once remote control ends.

DisplayRefreshTask runs the opposite direction, writing to the interface board's SIPO shift registers whenever an internal variable named DisplayRegister changes. Each Execute() call compares the current value against the value last written and, only on a difference, clocks the new 16 bits out MSB-first, high byte then low byte, over the ESP32's hardware SPI bus, toggling the latch line before and after with a configurable settle time to let the shift register outputs stabilize.

BT201 Driver

The BT201 Bluetooth/MP3 module is controlled over a UART using AT commands, one at a time, via BT201TransactionTask, a Task subclass that drives a small state machine: send a command, wait for its acknowledgement, then, for track playback, wait for the MV/MD/MO completion sequence. Each command is paired with a response template describing the expected reply so incoming lines can be matched automatically. Reception happens in a separate BT201ReceiverTask, which classifies each incoming line as either a synchronous reply (pushed directly to the transaction task) or an asynchronous, unsolicited message (fired to subscribers, which is how BLE-simulated switch presses reach SwitchScanTask). While a track plays, out-of-band commands such as stop, pause, fast-forward, and rewind are queued separately so they can interrupt playback without disturbing the main command queue. BT201ParentTask is a small base class that application tasks derive from when they play audio, so a mode switch always waits for playback to stop cleanly before the next mode begins (inheriting from BT201ParentTask provides the shutdown handshake sequence behavior mentioned previously). A separate BT201Provisioning utility handles the one-time flash configuration of a new or factory-reset chip (device name, volume, playback mode, and so on) and is not part of the normal runtime path.

Audio Visualization (VU Meter)

VUMeterTask samples the BT201's SPK+ signal via the ESP32's ADC, driven by a hardware timer ISR at a fixed rate, into a sliding window buffer. Once a window fills, an RMS envelope is computed with DC removal and a zero-crossing/activity check that rejects noise and silence.

The envelope is then smoothed with an asymmetric charge/discharge model that mimics an analog VU meter's ballistics. Attack and decay are flash-configurable time constants (0.1 ms units, 0 = instant), set separately for the music styles (vuAttack, vuDecay) and the voice styles (voxAttack, voxDecay), with a fast attack and slower decay by default. On the binary style, a slow attack makes the display behave like a counter. An adaptive threshold is tracked alongside for the voice-activity styles.

The resulting envelope value drives several selectable display styles (classic left-to-right/right-to-left bar graphs with peak hold, a center/edge symmetric bar, voice-activity styles including one modeled on the B9 robot from Lost in Space, and a raw binary display), each mapping the envelope to a 16-bit LED pattern through a precomputed lookup table. A one-time calibration routine measures and stores the ADC's DC offset in flash.

Appendix I: Task Reference

The full set of cooperative tasks registered with the scheduler, referenced from the Firmware Overview above.

System Tasks

Task	Description
`Blink`	Blinks the onboard status LED as a heartbeat indicator.
`DisplayRefresh`	Writes updated LED state to the interface board's SIPO shift registers via hardware SPI (falling back to software shiftOut() if disabled) whenever DisplayRegister changes.
`SwitchScan`	Reads and debounces the panel's PISO-registered toggle and sense switches, firing subscriber callbacks on change.
`DisplaySwitches`	Echoes the address/data switch states directly to the LED display (Switch Echo mode).
`BT201Receiver`	Reads and classifies incoming BT201 UART lines as synchronous replies or unsolicited async messages.
`BT201Transaction`	Drives the BT201 AT-command state machine: sends commands, awaits acknowledgement, and tracks track-playback completion.
`Startup`	Runs once at power-up: initializes the BT201, plays an intro track, then hands off to UITask.
`UI`	Main operating-mode task started once Startup completes; hands day-to-day panel control to the user.

Panel Modes

Task	Description
`Flash`	Front-panel interface for viewing and editing persistent flash (NVS) settings via the switch register, with a hidden provisioning function and confirmation tones.
`VUMeter`	Samples BT201 audio via the ADC and drives audio-reactive LED display styles (bar graphs, B9-style, and others).
`PanelLock`	Self-destruct-themed lockout sequence triggered by the panel-lock switch: radiate, override-countdown, and final-countdown display phases synced to an audio track, ending in a damaged-radiate effect and optional reboot.
`Clock`	DS3231 RTC-based clock: displays time, temperature, and a countdown timer, with quarter-hour chimes and per-second ticks.
`KnightRider`	Full Knight Rider–themed experience: scanning display pattern paired with an audio track.

Task	Description
`Computer`	Cycles LED display patterns synchronized with audio playback from the COMPUTR track folder, in random or manually selected pattern/track modes.
`Bluetooth`	Puts the BT201 into Bluetooth audio mode with auto-switch and drives the VU meter; EXAMINE cycles the display style.
`Jukebox`	Folder-based music playback with an integrated VU meter; folder and track selection via the switch register, with random shuffle or sequential play.
`Soundboard`	Plays themed sci-fi/movie audio clips (Robby, B9, HAL 9000, Star Trek, Star Wars, Colossus) on a timed schedule, with optional idle LED effects between tracks.

Display Effects

Task	Description
`KnightRiderDisplay`	Visualization-only Knight Rider scanning pattern (no audio).
`CrissCrossDisplay`	Blinkenlights display
`CheckerboardDisplay`	Blinkenlights display
`WigWagDisplay`	Blinkenlights display
`DiamondDisplay`	Blinkenlights display
`DissipateDisplay`	Blinkenlights display
`ChaserDisplay`	Blinkenlights display
`RadiateDisplay`	Blinkenlights display
`SnakeDisplay`	Blinkenlights display
`ImplodeDisplay`	Blinkenlights display
`Implode2Display`	Blinkenlights display
`SplitDisplay`	Blinkenlights display
`JoinDisplay`	Blinkenlights display
`SqueezeDisplay`	Blinkenlights display
`MarquisDisplay`	Blinkenlights display
`ChopsticksDisplay`	Blinkenlights display
`PanelDisplay`	Blinkenlights display
`OctalCountDisplay`	Blinkenlights display
`OctalFillDisplay`	Blinkenlights display
`PulseDisplay`	Blinkenlights display
`IdleDisplay`	Blinkenlights display
`BinaryCounterDisplay`	Increments a 16-bit binary counter each cycle and displays it directly.

Task	Description
`UpDownCounterDisplay`	Runs independent up and down 8-bit counters, packed into the high and low display bytes.
`RandomDisplay`	Displays a new random 16-bit pattern each cycle.
`RandomBitFillDisplay`	Randomly fills all 16 bits one at a time, then randomly clears them, alternating between fill and clear phases.
`OverrideCountdownDisplay`	Binary countdown from a configurable start count, collapsing into a shrinking bar near zero.
`FinalCountdownDisplay`	Binary countdown from 30 seconds, collapsing into a shrinking bar near zero.
`DamagedRadiateDisplay`	Radiating pattern with individual LEDs progressively and randomly “killed” on a schedule, simulating panel damage.

Appendix II: BT201 Musings

The BT201 sound card turned out to be one of the better discoveries on this project. It is a small, inexpensive module that switches cleanly between SD card audio and Bluetooth audio, which is exactly what the panel needed. It supports some degree of flash customization, for example the silencing of built-in audio prompts. It has also opened the door to something bigger: its BLE/SPP passthrough lets commands flow to and from the panel over Bluetooth, which is the basis for a planned phone app. These notes capture some observations on working with the module, expanding on the BT201 Driver summary above.

Reading the Documentation

The official BT201 manual is available online as a PDF: https://www.fabian.com.mt/viewer/39404/pdf.pdf. It contains a fair amount of information, but is often light on detail, and the translation from the original Chinese can be opaque in places. Two things help work around that.

- Feeding the manual to Claude.ai and asking specific questions, or having it produce summarized tables of the command set.
- Writing small exploratory test code: send a command, then poll for a response with a timeout, to see exactly what each command replies with.

Most commands acknowledge with a simple OK. Some follow up with something more specific, such as QM+3. Playing an audio track responds with OK immediately, and then, once the track finishes, with MV, MD, and MO in that order, provided Auto Return Data has been set in flash (see Flash Provisioning below).

Command and Response Protocol

It is possible to poll the BT201 to find out when a track has finished playing, but it is simpler to let the module report it, since it sends the completion sequence above on its own (a disadvantage of polling is that it requires serial communications on each poll request). Other commands switch the active audio source among SD, Bluetooth, and so on. The driver needs to be written carefully to keep the command stream synchronized with the application: all communication with the BT201 must be non-blocking, handled by polling for responses, and commands are typically sent one at a time, with no more than one in flight.

There is one exception to the one-command-at-a-time rule. Some commands are designed to interrupt whatever is already running, such as stop, pause, fast-forward, and rewind while a track is playing. These are treated as "out of band" (OOB) commands. Before sending one, the driver first confirms that a play command is actually in flight, that is, that it has been sent and no end-of-track response has been received yet.

Task Architecture Notes

The BT201 Driver section above describes BT201TransactionTask and BT201ReceiverTask at the architecture level. A bit more detail on how the two cooperate: the transmitter task handles both primary commands, like play track, and OOB commands, through two separate queues. Primary commands enter a front-end queue that is serviced whenever the transmitter is not already busy with a command in flight. OOB commands enter a separate queue that is serviced only while a play command is in flight, waiting on its end-of-track response. Each command is wrapped in a small transaction object encapsulating the command string and its state (inactive, pending, or

complete), and the transmitter also handles faults: if an acknowledgment is not received within the timeout period, a timeout error is raised.

The receiver task reads serial data from the BT201 (Serial2, in this case). Each response ends with a carriage return and line feed. Once an end of line is detected, the receiver checks whether the line is actually an expected response: a line beginning with OK, MV, MD, MO, or QM is categorized as a response and queued, while an ER (error) is logged. Anything else is treated as an unsolicited, asynchronous message from the BT201, handled through the subscription model mentioned above; the BT201 itself uses Q* and T* prefixes for these unsolicited messages, such as mode changes or Bluetooth connect and disconnect events. The receiver also runs its own timeout: the timer resets on every character received and concludes once the end of line arrives.

When the transmitter task runs and has active transactions, at most two at a time (for example, a play command and a fast-forward OOB command), it checks the receiver's response queue. If it is not empty, it dequeues the next response and tries to match it against one of the active transactions. A match completes that transaction; a response that matches neither is discarded and logged as an error. Both tasks are structured as state machines: whatever is waiting in the queues, whether transactions to process or responses to match, drives the next state transition, which keeps the whole exchange controlled and synchronized.

BLE/SPP Command Passthrough

The BT201 also supports Bluetooth passthrough, which this project uses for remote operation. It advertises this capability under the device name BT201-BLE, separate from BT201-Audio (a.k.a. "PDP-11/05" since that is the name set following provisioning) used for audio streaming. Any command sent to the BT201 that it does not recognize as one of its own AT commands is simply forwarded out over the BLE link, and the same holds in reverse: non-AT commands sent from the ESP32 go out to the BT201, which passes them on to the connected BLE client, in this case a phone app.

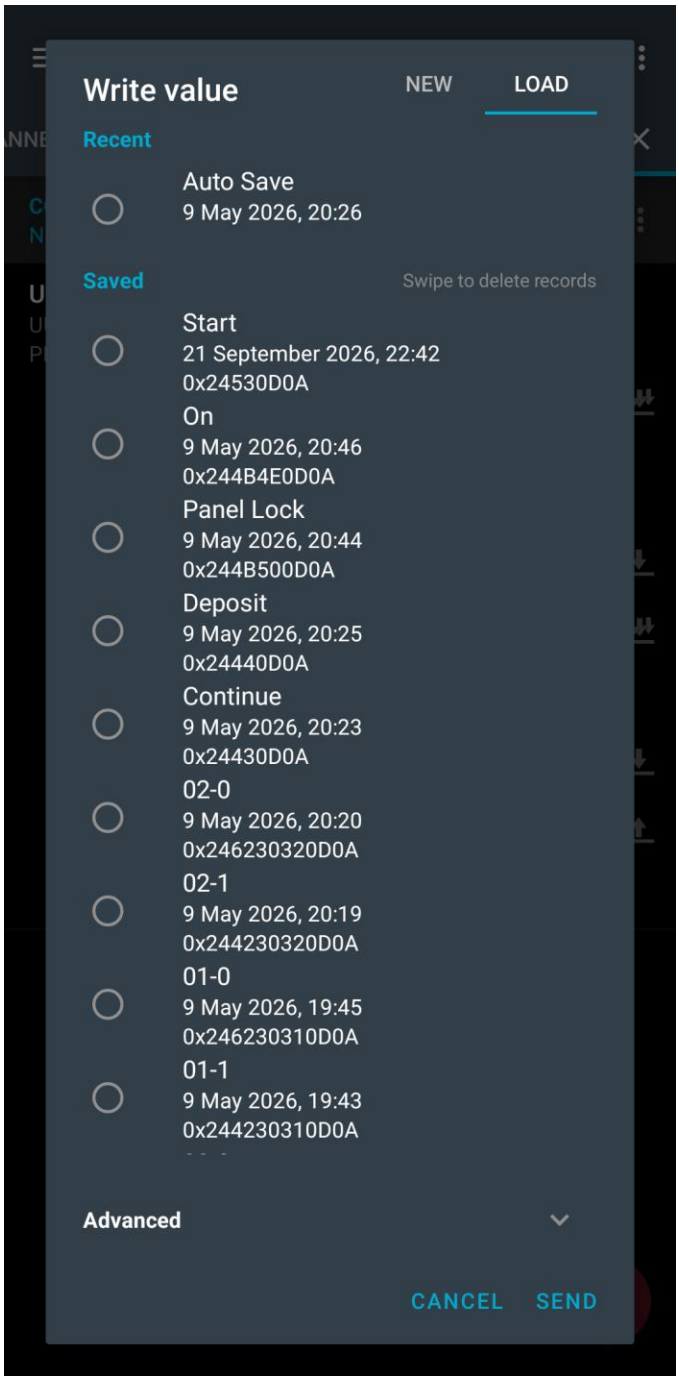
That symmetry means the phone app both sends commands to the panel and receives messages back from it, some solicited and some not. If the app requests the current switch states, the request travels down to the BT201, out the serial link to the ESP32, and the response travels back the same way. But if a switch is toggled on the panel with no request involved, the ESP32 sends that update on its own, and the app receives it as an unsolicited message, the same mechanism used internally by the receiver task's subscription model.

The nRF app on Android is useful for testing the BLE channel directly. Sending the bytes 0x24, 0x0D, 0x0A (properly terminated with a carriage return and line feed) shows up on the receiver task as an unsolicited async message, since it does not match any expected reply token, and gets echoed to the serial logging window. The eventual plan is a dedicated panel simulator app, built in Flutter, that uses this same channel to read panel state and send commands to it.

SwitchScanTask recognizes a small set of these commands for simulating physical switch input over BLE:

Command	Action	Notes
`$Bnn / $bnn`	Set or clear switch bit nn	Uppercase B sets the bit (switch on); lowercase b clears it (switch off). nn is a two-digit bit number, 00–15.
`$L`	Pulse LOAD ADDRS	Momentary; case of the letter is ignored.
`$E`	Pulse EXAMINE	Momentary; case ignored.
`$C`	Pulse CONTINUE	Momentary; case ignored.
`$S`	Pulse START	Momentary; case ignored.
`$D`	Pulse DEPOSIT	Momentary; case ignored.
`$H / $h`	Set ENABLE/HALT	Uppercase H enables (RUN); lowercase h halts.
`$KP`	Engage panel lock	Same effect as turning the physical keyswitch to the lock position.
`$KN / $KO`	Release panel lock	Any value other than P releases the lock.
`TL+02`	BLE disconnect notification	Restores the switch and sense registers to the panel's actual physical state.

nRF Connect's Write Value dialog can save these byte sequences as named presets, which makes it convenient to fire off individual switches or sense-switch pulses while testing without retyping them each time.



nRF Connect's Write Value dialog, with saved presets for several of the commands above.

Flash Provisioning

This expands on BT201Provisioning, mentioned in the BT201 Driver section above. Flash settings are configured in two phases, since the factory reset needs a power cycle before it takes effect. Phase one issues the factory reset, AT+CW, and then waits in an idle loop; the BT201 must be power-cycled after this before continuing. Phase two, once the reset has taken hold, sends the specific settings the application needs. Both phases use straightforward blocking sends and polls, since provisioning is normally a one-time operation, with an extra one-second delay added after each command since these are writes to flash. It may be possible to collapse the two phases by issuing a soft reset (AT+CZ) with a few seconds' delay in place of the power cycle, though that has not been tried yet.

The settings currently in use:

Setting	Command	Description
Mute Audio Prompts	AT+CN00	Mutes tone and voice prompts on play, connect, and similar events.
Quiet Power-Up	AT+C201	Minimizes serial output during power-up (only QM+09 is emitted).
Set BT Audio Name	AT+BD	Sets the Bluetooth audio device name as it appears to pairing clients.
Wait for Mode Instruction	AT+CP01	Waits for an explicit mode instruction on startup rather than auto-selecting a source.
Auto Return Data On	AT+CR01	Enables the OK / MV / MD / MO completion sequence used to detect end of track.
Single Play	AT+CJ01	Plays a single track rather than looping it.
Set Initial Volume	AT+CA	Sets volume (0–30). Can also be set at runtime and takes effect immediately; Bluetooth audio may override the current value.
Enable BT in Background	AT+CG01	Needed for BLE/SPP to operate.

Flash Persistence Reference

Notes extracted from the manual on which command settings persist across power cycles:

Command	Function	Persists at Power-Off?	Basis
AT+CGxx	BT background	Yes	Will not take effect until the power is cut off.
AT+B3xx	A2DP audio	Yes	Implied by "next power-on after setting is valid" (same section as B4/B5).
AT+B4xx	BLE on/off	Yes	Next power-on after setting is valid.
AT+B5xx	EDR on/off	Yes	Same, next power-on is effective.
AT+CAxx	Volume	Yes	Chip will automatically memorize when power fails.
AT+CPxx	Power-on mode	Yes	Requires the next power-on to work.

Command	Function	Persists at Power-Off?	Basis
AT+CNxx	Prompt tone	Yes	Chip will be powered down and saved.
AT+CKxx	Auto-switch BT	Unclear	Manual doesn't explicitly state.
AT+CMxx	Current mode	No	Use AT+CP for a persistent power-on mode instead.
AT+CQxx	EQ	No	Setting is not saved; every power-on reverts to default.

In short, AT+CG01 and AT+B300 belong in one-time initialization, while AT+CM04 and AT+CK00 are set at runtime, since the current mode (CM) is not persistent and the persistence of CK is unclear.

Appendix III: The Saga of the Missing Key

The key to the panel lock was missing, so a replacement had to be found. The following is from the gunkies.org PDP-11/05 page (https://gunkies.org/wiki/PDP-11/05):

> *"Unlike all the other keyed PDP-11s, which use a cylindrical Ace key, the /05's (and /10s) use a normal flat Yale-type key. The original key is a Chicago Lock Company key, code "GRB 2"; this is cut 215, on a Chicago K5K or Ilco S1041T blank. (It has been confirmed that a locksmith was able to produce a working key from this information.) If simply duplicating an existing key, Hillman Y11 and FR4 blanks may be used (both work, but one has to be trimmed a bit, length-wise)."*

I found an old office key that fit the lock and planned to file it to match the barrel lock's bitting. With no original key to compare against, I posted a request on the PDP-11 Facebook group for a close-up photo of one. Two images were posted.



Two key images received from the PDP-11 Facebook group. Left: a Hillman FR4 key. Right: a key photographed beside a ruler.

Using those images, I made a paper template and placed it over the office key as a guide for trimming. I used a Dremel and a hand file to match most of the bitting. Some low spots remained due to the office key's original cuts, so I filled them in with solder.

The result was a working key.



The office key after filing, with solder filling the low spots.

With a working key in hand, I visited my local ACE Hardware store looking for a suitable blank. The closest match was a Hillman E80. It has the same keyway but is longer. After cutting, the bitting matched my office key. The length was easy to trim with a Dremel cutoff disk and smooth with a hand file.





Left: the Hillman E80 key card. Right: the finished key after cutting and trimming to length.

I now had a working key cut on a proper key machine. Soon after, someone who read my post offered to sell me an official key. I no longer needed one, but I liked the idea of having it, so I bought it. It is stamped GRB 2.



The official GRB 2 key.

I then found a lot of 10 Ilco S1041T blanks on eBay and bought those as well. I brought them and the official key to a locksmith, who then produced the duplicates.



Left: an Ilco S1041T blank. Right: the duplicated keys.

What began as one missing key ended with a ring full of them! I now have more PDP-11/05 keys than I will ever need, so I plan on making them available to others who may need them in the future.